AETHER Platform — Solution Details & Agentic Architecture

Classification: *Executive Summary & Platform Technical Specification*

Target Audience: *CIO, CAIO, CISO, Lead Enterprise AI Architects*

Status: *Release Technical Specification (v3.0.0)*

---

1. Executive Summary & Solution Overview

The Enterprise Challenge: The Invisible AI Workforce

As enterprises aggressively adopt AI agents to automate business processes, they face an governance crisis. AI agents are being built across heterogeneous platforms—**Microsoft Copilot Studio**, **Salesforce Agentforce**, **AWS Bedrock**, **Google Vertex AI**, and custom runtimes utilizing **LangGraph**, **CrewAI**, or **AutoGen**.

This distributed model results in:

- **Shadow AI:** IT has no inventory or visibility into where agents run or what credentials they use.
- **Lack of Governance:** Agents access PII, execute financial transactions, and transfer data across regions without policy enforcement.
- **Regulatory Risk:** Compliance gaps against frameworks like the EU AI Act, NIST AI RMF, ISO 42001, SOC 2, and GDPR.
- **Runaway Costs:** Unmonitored token consumption and API licensing charges scattered across billing portals.

AETHER: The Enterprise AI Workforce Control Plane

AETHER (Autonomous Enterprise Technology for Harmonized, Ethical, and Responsible AI) is the unified control plane, system of record, and monitoring engine for the enterprise AI workforce. Think of AETHER as **"Workday for AI Workers"** and **"Visa for AI Agents."**

AETHER establishes:

1. **A Unified Inventory & Registry:** A single pane of glass for discovering, cataloging, and managing AI workers.
2. **Autonomous Supervisory Agents:** A workforce of 11 AI agents that continuously monitor and optimize the enterprise AI portfolio.
3. **Active Governance & Enforcement:** Real-time policy evaluation and automated compliance mapping.

4. **SQLite System of Record:** Dual-persistence engine tracking active and deboarded agents to ensure cumulative historical audibility.

AETHER Platform Architecture Flowchart

[Visual component: refer to solution-details.md or AETHER Command Center UI for interactive rendering]

2. The Multi-Agent Supervisory Architecture

AETHER governs the enterprise AI workforce by deploying a dedicated workforce of **11 autonomous supervisory agents**. These agents analyze the telemetry, behavior, and configurations of registered AI workers.

The 11 Supervisory Agents Specification

Icon	Agent Name	System Purpose	Trigger / Schedule	Primary Outputs	Key Metrics Tracked
■	**Discovery Agent**	Continuously scan external platforms via connectors to find unregistered AI agents.	Continuous (300s)	Registered Inventory, Discovery Coverage Score, Discovery Events	Total scans, discovered agents, scan coverage, avg scan duration
■	**Identity Agent**	Validate ownership, accountability, sponsor mapping, and credentials rotation for all agents.	Continuous (600s)	Identity Score (0-100), Orphan Agent Alerts, Credential Rotation Warnings	Avg identity score, orphan agents count, unauthorized agents, pending rotations
■■	**Governance Agent**	Enforce the 8 enterprise AI policies, trigger approval workflows, and initiate blocking/escalations.	Continuous (60)	Governance Score, Policy Violations, Enforcement Actions (Block/Notify)	Policies evaluated, active violations, automated block actions, escalations
■	**Compliance Agent**	Continuously maps agent configurations to regulatory control frameworks and audits gaps.	Hourly	Compliance Score, Audit Readiness, Gap Analysis Reports	Controls evaluated, controls passed, open findings, framework-specific compliance (SOC2, GDPR, HIPAA, etc.)
■■	**Risk Agent**	Perform LLM-based multi-factor risk assessment, computing risk scores and classifying threat level.	Continuous (300s)	Risk Score (0-100), Risk Level (Low-Critical), Reasoning, Risk Factors	Agents assessed, critical/high/medium/low risk counts, avg risk score, risk trend
■	**Cost Agent**	Monitor token budgets, compute consumption, licensing costs, and generate optimization suggestions.	Continuous (300s)	Cost per Agent, Cost per Task, Department Spend, Optimization Recommendations	Total monthly spend, avg cost per agent, avg cost per task, budget utilization, savings identified
■	**Reliability Agent**	Track operational uptime, query latency, failure rates, and detect service degradation or outages.	Continuous (60)	Reliability Score, Incident Alerts, SLA Compliance, Health Dashboard	Avg availability, avg success rate, avg latency, active incidents, degraded agents count
■	**Observability Agent**	Ingest, parse, and correlate logs, events, metrics, and trace spans across all agents.	Continuous (30s)	Agent Health Graph, Telemetry Dashboard, Anomaly Alerts	Events collected, logs processed, traces analyzed, anomalies detected

Icon	Agent Name	System Purpose	Trigger / Schedule	Primary Outputs	Key Metrics Tracked
■	**Benchmarking Agent**	Compare agent economics and performance against peers, departments, and industry averages.	Daily	Peer Rankings, Benchmark Scores, Optimization Recommendations	Agents benchmarked, departments ranked, peer variance metrics
■	**Auditor Agent**	Act as chief internal auditor, summarizing findings, evaluating evidence, and generating remediation schedules.	Daily	Synthesized Audit Reports, Executive Findings, Remediation Schedules	Audits completed, open findings, critical findings, avg remediation days
■	**AI Workforce Manager**	Act as Chief AI Workforce Officer, generating executive briefings and strategic workforce forecasts.	Daily	Daily Executive Briefings, Weekly Governance Reports, Budget Forecasts	Briefings generated, reports published, recommendations issued, implementation rate

3. How KPIs are Programmatically Calculated

KPIs are calculated by the data engine (`js/data.js` via the `computeKPIs` function) using a hybrid approach. It checks for **Live LLM Analysis Results** cached from the Python backend server (`server.py`) and falls back to **Agent Configuration Metadata** if the backend hasn't been run.

In the executive C-Suite dashboards, AETHER defaults to **Cumulative (SQL) Mode**, fetching cumulative statistics from the backend `/api/db-stats` SQLite database. The user can toggle to **Active Fleet Mode** to display live metrics of only active agents.

C-Suite Cumulative vs Active Formulas

A. Total Onboarded Agents

- **Cumulative Mode:** Pulls from `dbStats.cumulative_onboarded`.

```
N_cumulative = N_active + N_deboarded
```

- **Active Mode:** Sums currently registered active agents.

```
N_active = sum A[i] quad (where A[i] = 1 for each active agent)
```

B. Cost & Budget Efficiency

- **Cumulative Spend:** Sums the monthly costs of both active and deboarded agents.

```
Spend_cumulative = sum[j] in active monthlyCost[j] + sum_k in deboarded monthlyCost_k
```

- **Cumulative Tasks:**

```
Tasks_cumulative = sum[j] in active tasks[j] + sum_k in deboarded tasks_k
```

- **Cumulative ROI (Labor Savings vs Operational Cost):** Estimates human labor offset at 0.10$ per task executed:

```
ROI_cumulative = frac(Tasks_cumulative times $0.10) - Spend_cumulativeSpend_cumulative times 100%
```

- **Average Cost Per Task (CPT):**

```
CPT_cumulative = fracSpend_cumulativemax(1, Tasks_cumulative)
```

- **Average Cost Per Agent (CPA):**

```
CPA_cumulative = fracSpend_cumulativemax(1, N_cumulative)
```

- **Budget Efficiency Score:**

```
Budget Efficiency_cumulative = minleft(100, roundleft( fracTasks_cumulative times
$0.10Spend_cumulative times 100 right)right)
```

4. How Agents Onboard Agents: Programmatic Pipeline

Onboarding is the process of registering and setting up monitoring for an AI agent. It runs through a pipeline defined in `AETHER.config.onboardAgent` within `js/agents-config.js`.

The Step-by-Step Onboarding Pipeline

Step 1: Input Validation & ID Generation

The interface passes the agent form data to `onboardAgent(agentData)`. The engine validates that `name` is present. If no `id` is supplied, it automatically generates a serial identifier:

```
Next ID = "AGT-" + String(max(Existing IDs) + 1).padStart(4, '0')
```

Duplicate checks are conducted; if the agent ID already exists, the request defaults to `updateAgentConfig()`.

Step 2: Platform and Monitoring Method Mapping

The pipeline maps the agent to its platform connector:

- Checks `platformConnectors` for a name match.
- Resolves `monitoringMethod` based on platform capability:
- **Microsoft integrations:** default to `api-poll`
- **LangGraph / Custom:** default to `sdk-push` or `hybrid`
- Determines endpoints (e.g. appending `/health` and `/metrics` to the agent base URL).

Step 3: Creation of the Standardized Registry Entry

A unified JSON schema is generated for the agent, combining identity, deployment topography, risk defaults, and operational history.

Step 4: SQLite Database Sync

The onboarding entry is mirrored to the backend server via `POST /api/agents`. `server.py` automatically updates the `agents` table in `aether.db` and records the initial state in the `kpi_history` database table.

Step 5: Pub/Sub Event Broadcast

The system publishes an `AgentOnboarded` event to the `AETHER Event Bus`. Subscribers like the Command Center Activity Feed, the Governance Auditor, and Slack notification channels react instantly.

---

5. SQLite System of Record & Cumulative Analytics

To fulfill requirements for regulatory audibility and workforce tracking, AETHER integrates a SQLite database (`aether.db`) containing three core tables.

1. Database Schema

A. `agents` Table

Stores currently registered active agents. Columns mirror the JSON registration schema:

- `id` (TEXT, PRIMARY KEY)
- `name` (TEXT), `platform` (TEXT), `department` (TEXT), `owner` (TEXT)
- `monthlyCost` (REAL), `tasksExecuted` (INTEGER), `riskScore` (INTEGER)

- `status` (TEXT), `availability` (TEXT), `avgLatency` (REAL)
- `onboardedAt` (TEXT), `lastSeen` (TEXT)

B. `deboarded_agents` Table

Stores archived profiles of all decommissioned AI agents:

- `id` (TEXT, PRIMARY KEY)
- `name` (TEXT), `platform` (TEXT), `department` (TEXT), `owner` (TEXT)
- `deboardedAt` (TEXT) - Timestamp of decommissioning
- `reason` (TEXT) - Deboarding reason (e.g., "Decommissioned", "Resource Optimization", "Policy Non-compliance")
- `monthlyCost` (REAL) - Spend at the time of deboarding
- `tasksExecuted` (INTEGER) - Cumulative tasks executed prior to deboarding

C. `kpi_history` Table

Logs a time-series record of the platform's overall security, financial, and operational posture:

- `id` (INTEGER, PRIMARY KEY AUTOINCREMENT)
- `timestamp` (TEXT), `total_agents` (INTEGER), `deboarded_count` (INTEGER)
- `monthly_spend` (REAL), `workforce_roi` (REAL), `high_risk_agents` (INTEGER)
- `cost_per_agent` (REAL), `cost_per_task` (REAL)

2. Interactive Deboarding & Event Purging

When an agent is removed via the UI or `DELETE /api/agents/<id>` API call:

1. **SQLite Archiving:** The agent's profile is read from the active registry and inserted into `deboarded_agents`.
2. **Active Removal:** The agent is deleted from `agents.json` and the SQLite `agents` table.
3. **KPI History Log:** `_record_kpi_history()` computes updated aggregates and logs a new historic entry.
4. **Zero-State Purging:** If the remaining active agent count drops to `0`, all transient events and results are cleared out of memory and database files, guaranteeing that the live activity feeds remain clean and unpopulated.

3. Clickable KPI Deep-Dives

Clicking a KPI card in cumulative mode routes the user to a detailed rationale page containing two contributor tables:

- **Active Agent Fleet Table:** Lists currently running agents contributing to the metric.

- **Deboarded Agents History Table:** Shows the SQLite deboarded archive containing prior cost and task contributions of decommissioned agents.

6. How AETHER Tracks Agent Activity & Performance

Agent tracking works through **Continuous Monitoring Cycles**, an **Event Bus architecture**, and **Live Agent Run Cycles**.

```
AETHER.events.emit() ■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ AETHER Event Bus ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■■■ Dashboard Log Feed ■■■ Alert Engine ■■■ Persistent Audit
Logs
```

1. The AETHER Event Bus

The event bus (`js/event-bus.js`) is the pub/sub core of the platform. It holds a rolling memory buffer of the last `500` events.

- **Simulation Mode (Auto-Play):** If auto-play is enabled, a background interval triggers every **8**-**12** seconds, generating mock activities (e.g. token spikes, model updates, policy violations) to demonstrate monitoring in real time.
- **Real-time Event Ingestion:** Connectors, SDK calls, and monitoring checks issue events (e.g., `PolicyViolation`, `AgentDiscovered`, `AgentOnboarded`, `AgentOffboarded`).

2. Live Agent Run Cycles (The Live Activity Feed)

When the user clicks the **"Run All Agents"** button in the Command Center:

1. A request is sent to the backend endpoint `/api/run-all`.
2. `server.py` runs the 6 analysis agents in sequence:

`Compliance rightarrow Risk rightarrow Governance rightarrow Cost rightarrow Health rightarrow Audit`

3. Each agent runs LLM analysis on the registered agents.
4. Once complete, `agents-runtime.js` processes the results and pushes **7 specific activities** into the Event Bus, naming each agent and detailing their findings.

7. Supported Platform Connectors (12 Integrations)

AETHER integrates out-of-the-box with **12 enterprise platforms and frameworks**, split across Microsoft ecosystems, hyperscalers, open-source development frameworks, and database runtimes.

ms-copilot	Microsoft Copilot	Microsoft	https://graph.microsoft.com/v1.0	OAuth 2.0 / Microsoft Entra ID	User.Read.All, Application.Read.All, AuditLog.Read.All, Reports.Read.All
ms-copilot-studio	Copilot Studio	Microsoft	https://api.powerplatform.com/v1	OAuth 2.0 / Entra ID	Environment.Read, Bot.Read.All, Bot.Read.All
azure-ai-foundry	Azure AI Foundry	Microsoft	https://management.azure.com/...	Azure Managed Identity / SPN	Reader, Contributor
ms-entra-id	Microsoft Entra ID	Microsoft	https://graph.microsoft.com/v1.0/identity	OAuth 2.0 / Entra ID Client Creds	Directory.Read.All, Application.Read.All, AuditLog.Read.All
openai-assistants	OpenAI Assistants	OpenAI	https://api.openai.com/v1	API Key / Bearer Token	assistants.read, threads.read, runs.read
azure-openai	Azure OpenAI	Microsoft	https://resource.openai.azure.com/openai	Managed Identity / API Key	Cognitive Services OpenAI User
langgraph	LangGraph	Open Source	https://api.smith.langchain.com/v1	API Key	runs.read, projects.read, traces.read
crewai	CrewAI	Open Source	https://api.crewai.com/v1	API Key	crews.read, agents.read, tasks.read
salesforce-agentforce	Salesforce Agentforce	Salesforce	https://contoso.my.salesforce.com/services/data/v62.0	OAuth 2.0 / JWT Bearer	api, chatter_api, einstein_analytics
databricks	Databricks Agents	Databricks	https://contoso.cloud.databricks.com/api/2.0	Service Principal / PAT	workspace.read, clusters.read, serving-endpoints.read
snowflake-cortex	Snowflake Cortex	Snowflake	https://contoso.snowflakecomputing.com/api/v2	Key Pair / OAuth	CORTEX_USER, DATA_READER
custom-agents	Custom Enterprise Agents	Custom / SDK	https://aether-api.contoso.com/v1/agents	AETHER SDK / API Key	agent.register, agent.heartbeat, agent.metrics

8. Governance Policies & Compliance Posture

The 8 Default Governance Policies Enforced

The Governance Agent evaluates agents against these 8 policies.

1. **GOV-PII-001 (PII Access Control):** Flags any agent processing PII (e.g. email, SSN, phone) without active privacy certification.
2. **GOV-FIN-001 (Financial Transaction Limit):** Flags agents handling financial operations that do not have multi-factor approvals configured.
3. **GOV-COST-001 (Cost Anomaly Detection):** Flags agents whose monthly token and runtime costs exceed their designated department budget.
4. **GOV-AUTH-001 (Unauthorized Agent Detection):** Flags agents discovered by scanners that do not have a registered owner or business sponsor.
5. **GOV-MODEL-001 (Model Version Control):** Flags agents utilizing unapproved foundation models (e.g. legacy model versions or unvetted external models).
6. **GOV-DATA-001 (Cross-Region Data Transfer):** Flags agents processing data across geographic boundaries (e.g. EU data routed to US-based LLM endpoints).
7. **GOV-CERT-001 (Certification Expiry):** Warns when an agent's annual architecture or compliance certification is within 30 days of expiry.
8. **GOV-IDLE-001 (Idle Agent Detection):** Flags agents with active statuses that have not executed a task or sent a heartbeat in over 30 days.

Framework Compliance Mapping

The Compliance Agent maps configurations and evidence to five key standards:

- **SOC 2 (Trust Services Criteria):** Security, confidentiality, and processing integrity controls for SaaS-deployed AI systems.
- **ISO 27001 (Information Security Management):** Access governance, logging, asset protection, and vulnerability management.
- **GDPR (General Data Protection Regulation):** User consent, data minimization, right to erasure, and cross-border processing limits.
- **SOX (Sarbanes-Oxley Act):** Internal control validation for financial AI calculations.
- **HIPAA (Health Insurance Portability and Accountability):** Containment of Protected Health Information (PHI).